

CityMind: Urban Intelligence System

Phase 2: Implementation Report

Course: Artificial Intelligence

Department of Computer Science

Zarrar Nadeem (24i-0735)

Shahmeer Zubair (24i-0580)

Raahim Babar (24i-0561)

Submission Date: 10th May, 2025

1. Introduction

CityMind is an AI-based urban simulation system built in Python using the Tkinter graphical framework. The city is modeled as a grid-based graph where each cell is a node connected to its four neighbors by road edges. The system applies five AI techniques to address five urban challenges: city layout planning using a Constraint Satisfaction Problem solver, road network construction using Kruskal's MST algorithm, ambulance placement using a Genetic Algorithm, dynamic emergency routing using A* search, and crime risk prediction using K-Means clustering combined with a K-Nearest Neighbors classifier.

All five modules share a single CityGraph instance. This means any change to the environment, such as a road flooding or a risk index updating, is immediately visible to every other module. The simulation runs for 20 steps by default and can extend if civilians are still pending.

2. System Architecture and Shared City Graph

The system is organized into nine Python files. The CityGraph class in cityGraph.py is the shared data structure that all modules read from and write to. The other modules are csp.py, mst.py, ga.py, astar.py, crime.py, simulation.py, appController.py, and ui.py.

Each node in the grid stores four properties: its type (Hospital, School, Industrial, Residential, PowerPlant, AmbulanceDepot, or Empty), its population count, a riskIndex float that acts as a travel cost multiplier, and an accessible boolean that marks whether the node can be entered. Each edge stores a base travel cost, a blocked flag that is set when the edge is flooded, and a built flag that is set when the edge is included in the road network. Standard edge costs are 1.0 and edges adjacent to Residential nodes cost 0.8. The weighted cost used by routing and placement is the base cost multiplied by the destination node's riskIndex, so changes made by the crime module automatically affect pathfinding without any extra wiring.

3. Challenge 1: City Layout Planning

The CSP module in `csp.py` places all buildings on the empty grid while satisfying three constraints: Industrial zones cannot be adjacent to Schools or Hospitals, every Residential area must be within a configurable number of road hops of a Hospital (default 3), and every Power Plant must be within a configurable number of road hops of an Industrial zone (default 2). Both hop limits are adjustable from the Constraints panel in the interface, allowing the user to test different rule configurations.

The solver uses backtracking search. Before the search begins, the list of buildings to place is sorted so that Hospitals and Industrial zones are placed first, since other building types depend on their positions for constraint checking. At each step the Minimum Remaining Values heuristic picks the building type with the fewest valid empty cells remaining, ensuring the most constrained type is handled first. For ordering the candidate cells, the Least Constraining Value heuristic scores each cell by temporarily placing the building there, counting how many valid cells remain for all other pending buildings, and then undoing the placement. Cells that leave the most freedom for other buildings are tried first. After each placement, Forward Checking verifies that every remaining building type still has at least one valid cell. If not, the branch is abandoned immediately.

If the full backtracking search fails, the solver falls back to a simple greedy placement that fills cells in order. A post-placement analysis then checks all three constraints and identifies which one has the most violations, reporting that specific conflict to the user in the event log.

4. Challenge 2: Road Network Optimization

The MST module in `mst.py` builds the road network using Kruskal's algorithm. All grid edges are sorted by travel cost. The algorithm processes them in ascending order using a Union-Find structure with path compression and union-by-rank, adding each edge to the spanning tree only if it connects two previously disconnected components. The result is a minimum-cost tree that spans every node. After the MST is built, all included edges are marked as built in the shared graph, enabling other modules to restrict movement to actual roads.

To satisfy the requirement of two independent emergency routes between the primary Hospital and the primary Ambulance Depot, A* search is run twice. The primary Hospital and Depot are selected as the nodes of each type that are closest to the center of the grid. Route A is found by running A* on the built MST edges. To find Route B, all edges belonging to Route A are temporarily flooded, and a second A* run is performed over all grid edges. This forces the discovery of a completely separate path that shares no road with Route A. After Route B is recorded, the temporary flood is removed and all Route B edges are added to the built set. Both routes are displayed as colored overlays in the interface.

5. Challenge 3: Ambulance Placement

The GA module in `ga.py` places three ambulances by running a Genetic Algorithm. Each chromosome is a list of three node coordinates, one per ambulance. Fitness is computed by running Dijkstra's algorithm from each ambulance position using weighted costs that include risk indices, then finding the maximum of the minimum distances from any accessible node to its nearest ambulance. A lower fitness value means a better placement. A fitness cache keyed on the sorted position tuple avoids recomputing the same chromosome across generations.

The algorithm runs for 60 generations over a population of 30 chromosomes. Each generation, the population is sorted by fitness and the top 15 are kept as parents. Fifteen children are produced by one-point crossover of adjacent parent pairs. A duplicate-fix step replaces any repeated positions in a child with unused accessible nodes. Each child then has a 10 percent chance of mutation, which replaces one position with a randomly chosen accessible node not already in the chromosome. The best chromosome seen across all generations is tracked separately and written to the graph at the end.

The GA runs once during setup. When the simulation's risk-shift event fires every five steps, a lightweight greedy repositioning is applied instead: each ambulance checks its built-road neighbors and moves to any neighbor that reduces its individual worst-case distance. This keeps coverage responsive to changing risk without re-running the full algorithm.

6. Challenge 4: Emergency Routing Under Changing Conditions

The A* module in `astar.py` routes a medical team from the primary Hospital to a list of randomly placed civilians. The team visits them in order. Each call to `findPath` runs A* with a min-heap priority queue. The cost $g(n)$ accumulates weighted edge costs and the heuristic $h(n)$ is the Manhattan distance to the goal, which is admissible on a grid and guarantees the shortest available path.

State is stored in a `RouterState` object that holds the team's current position, the list of civilian targets, the remaining steps in the active path, and lists of reached and skipped civilians. On each simulation step, the team advances one cell along the current path. If the next cell becomes blocked due to a flood, A* is called again from the current position rather than from the start, computing the best detour without restarting the journey. If no path to the current civilian exists at all, that civilian is added to the skipped list and the team moves on to the next target. The planned path is drawn as a dashed overlay on the map and updates every frame. Users can also inject new civilian emergencies at any time using the Emergency mouse tool, which appends a new target to the civilian list.

7. Challenge 5: Crime Risk Prediction

The crime module in `crime.py` runs a three-stage pipeline. The first stage extracts two features for every node: its population count from the graph and its BFS distance to the nearest Industrial zone. K-Means clustering with k equal to 3 is then run on this feature matrix. The three cluster centroids are ranked by average population density, and the clusters are labelled High, Medium, and Low accordingly.

The second stage generates synthetic labels using a formula. Each node receives a score that combines population density with 60 percent weight and a proximity-based value derived from the inverse BFS distance to Industrial zones with 40 percent weight. Scores above 66 percent of the maximum are labelled High, scores above 33 percent are Medium, and the rest are Low. The percentage agreement between the K-Means labels and the formula labels is computed and logged. In a typical run this is around 60 to 70 percent, confirming that both approaches capture similar patterns.

The third stage trains a K-Nearest Neighbors classifier with k equal to 5 on the full feature matrix using the formula labels as ground truth. The trained model predicts a risk label for every node. Each label is converted to a riskIndex multiplier (High 2.5, Medium 1.75, Low 1.25) and written back to the shared graph. From this point on, every call to `getWeightedCost()` incorporates the crime risk of the destination node, affecting both the A* router and the GA fitness evaluator automatically. The top 10 highest-risk nodes are also stored as police officer positions and rendered as badges in the interface.

8. Simulation Loop and Integration

The Simulation class in `simulation.py` runs the setup and the per-step loop. During setup, modules are called in order: CSP places buildings, MST builds roads, the crime module assigns risk indices, the GA places ambulances, and the A* router initializes. This ordering is necessary because each module depends on the state established by the previous ones.

Each simulation step runs four operations in sequence. First, with a user-configurable probability, one random unblocked edge is selected and flooded. Second, every five steps, three random non-empty nodes have their riskIndex increased by 0.25, capped at 3.0, and the greedy ambulance repositioning runs. Third, the medical team advances one step along its path. Fourth, any ambulance sitting on an inaccessible node is relocated to its nearest accessible neighbor.

The simulation runs for 20 steps. If civilians are still pending at step 20 it extends for up to 30 additional steps. When finished, the event log shows a summary of civilians reached, civilians skipped, roads flooded, and the CSP result.

9. User Interface

The interface is built in Tkinter and organized in three columns. The left column shows the city map canvas. The middle column holds all controls. The right column shows the legend and the event log.

Map Canvas

Cells are drawn using PNG sprites loaded with Pillow, with separate images for each building type, four ground variants chosen deterministically by cell position, and an animated tree sprite loaded from a strip and cycled every 100 milliseconds. The canvas resizes dynamically when the user changes the grid size. Three overlay modes can be toggled: Road Network colors each road by type (standard, residential discount, flooded, or unbuilt), Coverage shows a heatmap of Dijkstra distances from the nearest ambulance, and Crime Risk shows a gradient based on each node's riskIndex.

Controls

The Settings panel lets the user set grid size, per-building counts, flood probability, and step delay. The Constraints panel exposes the residential hop limit, power plant hop limit, and the industrial adjacency toggle. Pressing Generate City rebuilds the simulation from scratch with the current settings. The Mouse Tool panel switches between Inspect (click a cell to see its stats), Flood (click two adjacent cells to flood the road between them), and Emergency (click a cell to add a civilian target for the router).

Event Log

The event log on the right is a scrollable read-only text widget that records every system event in order, including CSP placement results, MST construction details, crime pipeline statistics, ambulance positions, flood events, risk shifts, rerouting decisions, civilian rescues, and the final simulation summary.

10. Work Breakdown

Zarrar Nadeem (24i-0735)

- Challenge 5: K-Means clustering, synthetic label generation, KNN classifier training, and risk index integration into the shared graph.
- Challenge 4: A* pathfinding, RouterState management, real-time rerouting on flood events, and civilian skip logic.
- eventLog.py: Scrollable live event log widget.

Shahmeer Zubair (24i-0580)

- Challenge 3: Genetic Algorithm for ambulance placement, Dijkstra fitness evaluation, and greedy repositioning on risk shifts.
- simulation.py: 20-step simulation loop, flood events, risk shift scheduling, and ambulance safety checks.
- ui.py: Full Tkinter interface, sprite loading, canvas rendering, overlay system, and animation loop.

Raahim Babar (24i-0561)

- cityGraph.py: Shared CityGraph class, node and edge management, cost functions, accessibility and flood handling.
- Challenge 1: Backtracking CSP with MRV, LCV, forward checking, greedy fallback, and conflict identification.
- Challenge 2: Kruskal's MST with Union-Find, dual A* route construction for Route A and Route B, and built-edge marking.

11. Conclusion

CityMind implements five AI techniques within a single system built on a shared city graph. Constraint Satisfaction handles city layout, Kruskal's MST and A* build the road network, a Genetic Algorithm places ambulances, A* handles dynamic emergency routing, and K-Means combined with KNN produces crime risk predictions that feed back into routing and placement. Because all modules operate on the same CityGraph instance, changes propagate automatically across the entire system. The implementation matches the design set out in the Phase 1 document and produces a working real-time simulation with a fully interactive graphical interface.

12. Screenshots

Figure 1: Initial state of the CityMind interface before a city is generated.

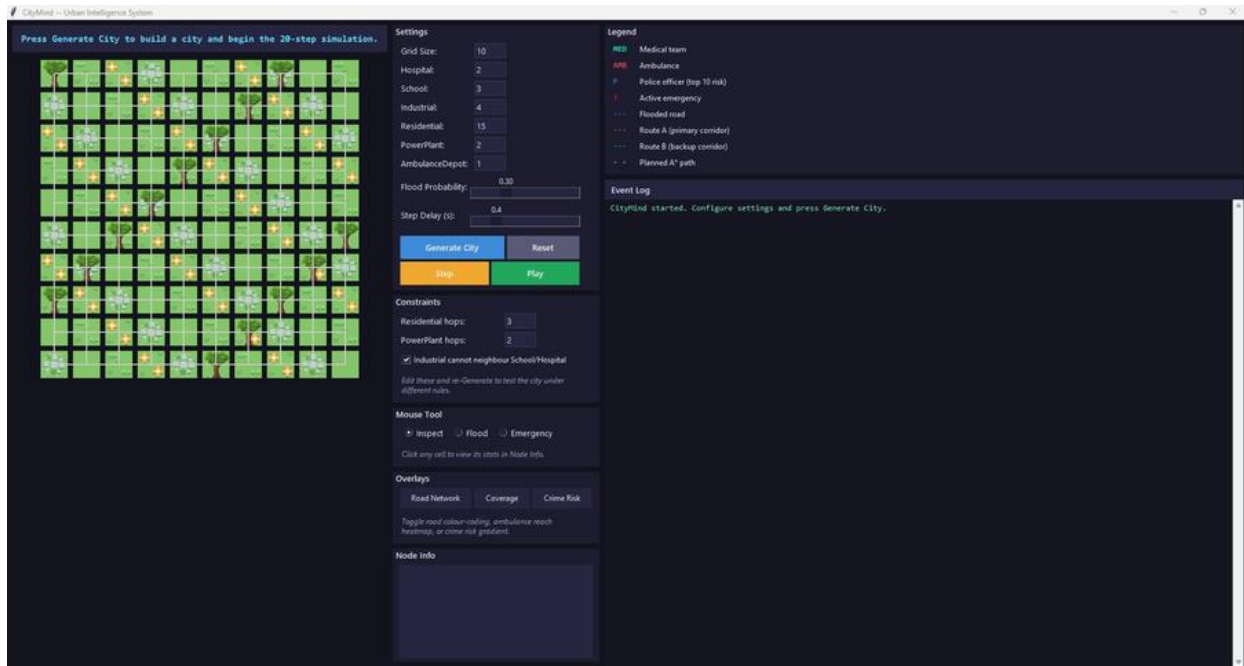


Figure 2: Simulation at step 5 of 20 showing buildings placed by CSP, the orange Route A overlay built by MST, the medical team navigating toward civilians, ambulances positioned by the GA, and police badges on high-risk nodes. The event log on the right records each system decision as it happens.

